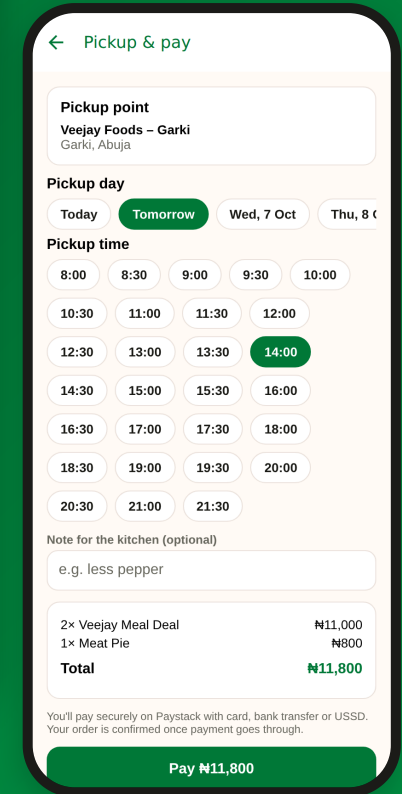
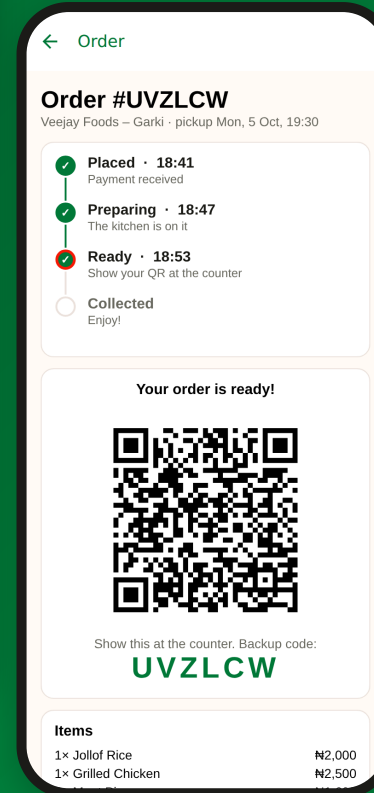
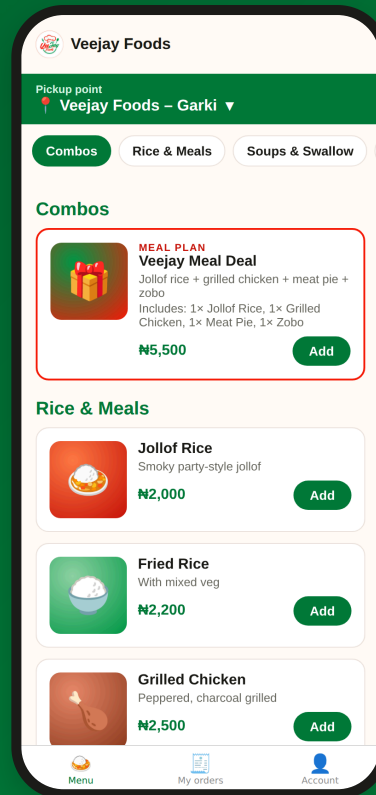




PRODUCT DESIGN & DELIVERY PLAN · V1.1

# Veejay Foods

Order ahead. Skip the queue. A mobile pre-order and pickup app for Veejay Foods in Abuja — pay with card, bank transfer or USSD, track your order, and collect with a QR code.



Prepared 5 October 2026 · Veejay Foods · Launch area: Abuja Municipal (AMAC) · Pickup points shown (Wuse II, Garki, Jabi) are samples  
Customer app (Expo / React Native) · API (Node, Express, Prisma, PostgreSQL) · Staff & admin dashboard (React)

All screenshots are captured from the real code running against the real API. Menu items, prices and customers shown are sample data.

## The problem

Veejay Foods serves homemade African meals, pastries and local cuisine. Like any popular outlet in a busy city it loses sales and goodwill at peak hours: long counter queues, orders shouted over a crowd, cash handling, and “where is my order?” with no answer. Customers abandon the queue; staff get overwhelmed in bursts and idle in between.

## The idea

Let customers choose a **pickup point and a time slot, pay in advance**, and walk in to **scan a QR code**. The kitchen sees a live, time-ordered queue, so demand is spread across slots instead of arriving in a rush. Money is settled before food is made, and every order has an auditable trail.

### One-line promise

**“Pick it, pay it, collect it — no queue.”**

### Launch area: Abuja Municipal (AMAC)

Pickup points are limited to Abuja Municipal Area Council districts — for example Wuse, Garki, Maitama, Asokoro, Jabi, Utako and the Central Area. Other areas and cities are out of scope for now; the data model already supports adding more pickup points later without code changes.

## Why it works for each side

### Customer

- No waiting in line
- Choose the exact pickup time
- Pay by card, transfer or USSD
- Live status + push notifications
- Wrong order? Report with a photo, get refunded

### Owner

- Sales by day, outlet and item
- Sold-out toggles per outlet
- Refund control, audit trail
- Menu, prices and hours without a developer

### Kitchen / counter

- Orders arrive already paid
- Slot capacity caps the rush
- Clear New → Preparing → Ready board
- Scan-to-collect, no disputes
- Works for lunch and evening peaks

### Proposed success metrics

- Median counter wait at pickup < 2 min
- Cart → paid conversion
- Refund rate < 2% of orders
- Repeat orders within 14 days

Targets to agree with the business — none measured yet.



## Amina — the customer

27, works in Wuse II, Abuja. Has 25 minutes for lunch, hates queues, pays by bank transfer or card on her phone. Patchy mobile data on the commute.

**Needs:** fast reorder, certainty about pickup time, proof of payment, a way to fix mistakes.



## Musa — counter staff

Runs the counter during the lunch and evening rush. Works on a shared tablet or phone, no time for menus.

**Needs:** one screen showing what to cook next, big buttons, a scanner that just works, a fallback when a phone dies.



## The owner / admin

Wants to change prices, hide a sold-out dish, add a future branch, and see yesterday's sales over breakfast.

**Needs:** control without a developer, trustworthy numbers, safe refunds.

## The core journey (customer)

1

### Sign in

Phone number + SMS code

2

### Browse

Menu, combos, per-outlet stock

3

### Cart

Quantities, running total

4

### Pick slot

Outlet, day, time

5

### Pay

Paystack: card · transfer · USSD

6

### Track

Placed → Preparing → Ready

7

### Collect

Show QR, staff scan

## If something goes wrong

- **Paid but screen didn't update** — the app asks the server to re-check with Paystack; the webhook is the safety net.
- **Phone dies at the counter** — staff type the 6-character order code instead of scanning.
- **Wrong or cold food** — report in-app with a photo; admin approves and the refund goes back to the original payment method.

## Design rules that follow from the users

- A user with a paid, uncollected order is **never signed out** — they always have their QR.
- Every critical button is  $\geq 50$  px tall; the counter UI works one-handed.
- Prices are integers in kobo end-to-end — no rounding surprises in ₦.
- Network failures never look like “logged out” or “payment failed”.

# Nine capabilities, all built

| # | FEATURE                            | WHAT IT DOES                                                                                                                                                                                           | HOW IT WAS VERIFIED                                                         |
|---|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| 1 | <b>Phone + OTP sign-in</b>         | 6-digit SMS code (hashed, 5-min expiry, 5 attempts, rate-limited). 15-min access token + rotating 30-day refresh token. Never ends a session while an order is active.                                 | <b>16 automated tests</b> incl. expiry, lock-out, replay, active-order rule |
| 2 | <b>Menu &amp; combos</b>           | Categories, items, combos / meal plans with components, images, prices in ₦, per-outlet “sold out” switches.                                                                                           | Browser-verified (app + dashboard)                                          |
| 3 | <b>Pickup points &amp; slots</b>   | Outlets with weekly hours (an optional “24 hours” setting exists), prep time, slot length and per-slot capacity; capacity enforced under concurrent bookings. Ready for more branches.                 | <b>4 + 21 tests</b> slot maths, capacity race, hold expiry                  |
| 4 | <b>Cart &amp; checkout</b>         | Choose outlet, day and time; server recomputes the price; unpaid orders hold the slot for 15 minutes, then release it.                                                                                 | Tests + browser-verified                                                    |
| 5 | <b>Paystack payments</b>           | Hosted checkout (card, bank transfer, USSD) — card data never touches our code. Webhook verifies HMAC-SHA512, amount, currency, reference; idempotent. App can reconcile via server → Paystack verify. | <b>14 webhook tests</b><br><b>live Paystack not yet exercised</b>           |
| 6 | <b>Order tracking + push</b>       | Placed → Preparing → Ready → Collected, polled in-app and pushed via Expo/FCM at every change.                                                                                                         | <b>5 state tests</b><br><b>push not yet on a device</b>                     |
| 7 | <b>QR pickup</b>                   | Unforgeable per-order QR (HMAC). Staff scan; only a paid, READY order at that outlet can be collected — once. Order-code fallback.                                                                     | Tests: forged, unpaid, double-scan, wrong outlet, race                      |
| 8 | <b>Complaints &amp; refunds</b>    | Report a wrong order with a photo; admin approves (full or partial) → Paystack refund to the original method → webhook finalises. Failed refunds reopen the complaint.                                 | <b>7 tests</b><br><b>live refund not yet exercised</b>                      |
| 9 | <b>Staff &amp; admin dashboard</b> | Live order board (with sound), QR scanner, menu / outlets / staff management, complaints & refunds, sales reports.                                                                                     | Browser-driven against the live API                                         |

Orders are pickup-only at launch. Delivery, promo codes and loyalty are deliberately out of scope (see roadmap).

## Colour — taken from the Veejay logo



**Veejay Green**  
#019947  
Logo green ·  
login, accents



**Deep Green**  
#017A39  
Buttons & text  
(AA contrast)



**Veejay Red**  
#F81A08  
Logo red ·  
highlights



**Deep Red**  
#C8140A  
Red text on white



**Warm White**  
#FFFAF5  
Canvas

Green and red are sampled from the logo itself. The pure logo shades sit at about 3.6:1 and 4:1 against white, so text and button fills use the slightly deeper variants, which clear WCAG AA ( $\approx 5.4:1$  and  $\approx 5.9:1$ ).

## Principles

- **Food first** — large images, price always in green, “Add” is the loudest thing on a card.
- **State is always visible** — a four-step timeline with a red ring on “now”; one status pill per order.
- **Calm money screens** — one total, one button, a plain sentence about Paystack.
- **Big targets** — 50 px buttons, 34 px steppers, chips instead of dropdowns for slots.
- **Honest empty/error states** — tell people what happened and what to do next.

## Components



Primary button · 14 px radius



Ghost button



14:00

Chips for day / time / category



Status pills

**MEAL PLAN**

**Veejay Meal Deal**

**#5,500**

Combo card · red border



**Preparing**

**Ready**

Timeline

## Type & spacing

System font stack (San Francisco / Roboto) for speed and native feel. Scale: 13 · 16 · 18 · 20 · 26 · 32. Spacing in 4 px steps; screens use a 16 px gutter; cards 14 px radius with a 1 px warm-grey border instead of shadows.

Brand name, logo and palette live in one theme file per surface (app, dashboard, API), so a future rebrand is a small change.

# From phone number to a full basket in four screens

## Pick a slot, pay securely, done

IN PREPARATION

# Always know where your order is — and prove it's yours

# When something's wrong, fixing it is easy

# One screen to run the rush

IN PREPARATION

10 / 19

# See the business, and fix mistakes safely

IN PREPARATION

# Run the menu and the outlet without calling a developer



# One source of truth for “is it paid?”

IN PREPARATION

14 / 19

# Sixteen tables, money in integer kobo

IN PREPARATION

## Designed for the ways money apps actually fail

# What's proven — and what isn't yet

IN PREPARATION



